


欢迎选修《数据结构》

- 课程编号：30240184
- 授课教师：邓俊辉、尹 霞、陈键飞
- 开课单位：计算机系

《数据结构与算法》DSA学什么？

线性结构

- 01.Introduction
  用尺子丈量
- 02.Vector
  向量的排序
- 03.List
  列表的排序
- 04.Stack+Quene
  各类应用

半线性结构

- 05.Binary_Tree
  树的遍历
- 06.Binary_Search_Tree
- 07.BST_Application
- 08.Advanced_BST
  各种树及中序遍历
- 09.Dictionary
  小容器放下大空间

非线性结构

- 10.Graph
- 11.Graph_Application
  图的遍历
- 12.PQ
  只关注极值的“树”
- 13.String
  最常见的串匹配
- 14.Sorting
  永恒的排序算法

朴素的向量和列表

2. 向量

02-A.抽象数据结构

02-B.无序向量

02-C.有序向量

02-D.起泡排序

02-E.归并排序

02-F. 位图

3. 列表

03-A.循位置访问

03-B.无序列表

03-C.有序列表

03-D.选择排序（轮换）

03-F.插入排序（逆序对）

03-F.归并排序（逆序对）

无序、有序、排序

3．列 表

03-A.循位置访问

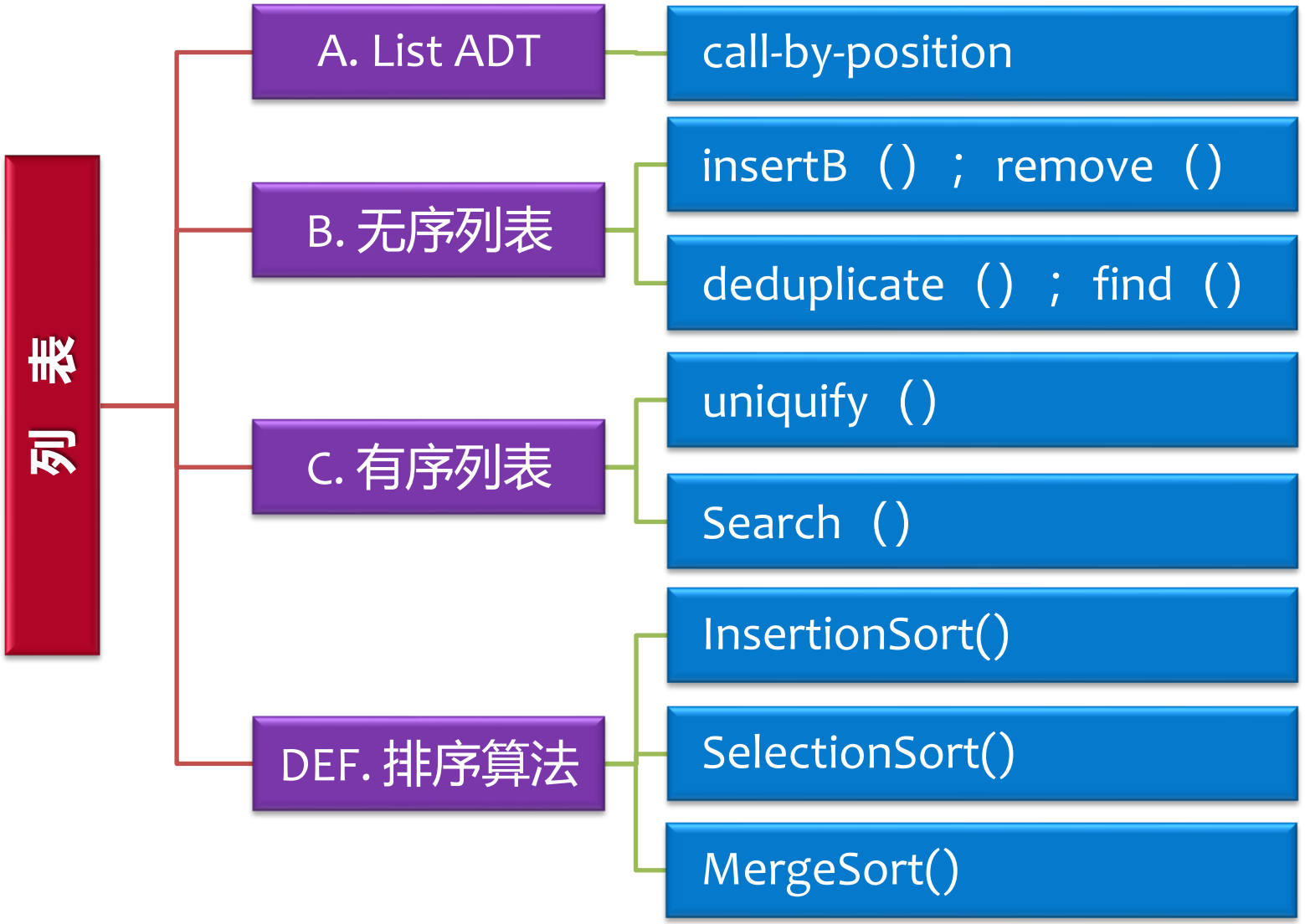
03-B.无序列表

03-C.有序列表

03-D.选择排序(轮换)

03-F.插入排序(逆序对)

03-F.归并排序(逆序对)



定义 → ADT接口 → 无序操作 → 有序操作 → 排序算法 → 扩展

算　法		VECTOR　(call-by-rank)	LIST (call-by-position)
扩容算法		倍增策略,分摊成本为O(1)	------
秩到位置		A[r]: O(1)	O(r)
无序	查找	find(e, lo, hi) : O(n)	find(e, n, Posi(T) p):O(n)
	插入	insert(r , e) : O(n-r)	insertB(Posi(T) p, e):O(1)
	删除	remove(lo, hi) : O(n-hi)	remove(p) : O(1)
	唯一化	deduplicate( ) : $O(n^2)$	deduplicate( ) : $O(n^2)$
	遍历	traverse( ) : O(n)	traverse( ) : O(n)
有序	唯一化	uniquify( ) : O(n)	uniquify( ) : O(n)
	查找	• BinSearch(A, e, lo, hi): O(1.5 log n) • FibSearch(A, e, lo, hi): O(1.44 logn) • 插值查找: O(loglogn)	search(e, n, Posi(T) p): 最好O(1)，最坏O(n)
排序	无序 → 有序	• BubbleSort: 最好O(n)，最坏$O(n^2)$ • MergeSort:　最坏情况下最优O(nlogn)	• InsertionSort(): • SelectionSort(): • MergeSort():

03.List

A-循位置访问

列表

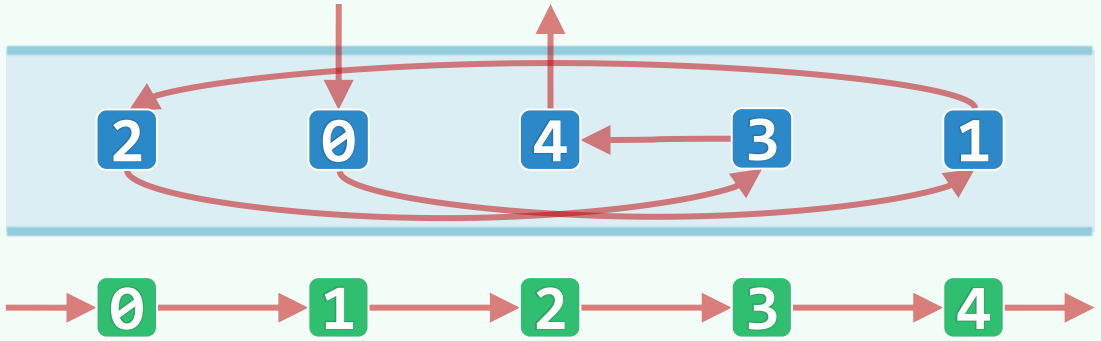
ADT：循位置访问

邓俊辉
deng@tsinghua.edu.cn

静态+动态|物理+逻辑

❖ 向量擅长于静态操作，但动态操作却显吃力

循秩访问：将元素的物理地址与其逻辑次序绑定



❖ 列表|list：向量的姊妹，都属于序列|sequence

仍按线性次序：$\mathcal{L} = \{ a_0, a_1, a_2, \ldots, a_{n-1} \}$；元素仍各有秩，相邻者互称前驱、后继

❖ 与向量不同，列表中各元素的物理次序不再与其逻辑次序直接相关

逻辑上紧邻的一对前驱与后继元素，物理上既不必保持前后次序，更未必相互紧邻

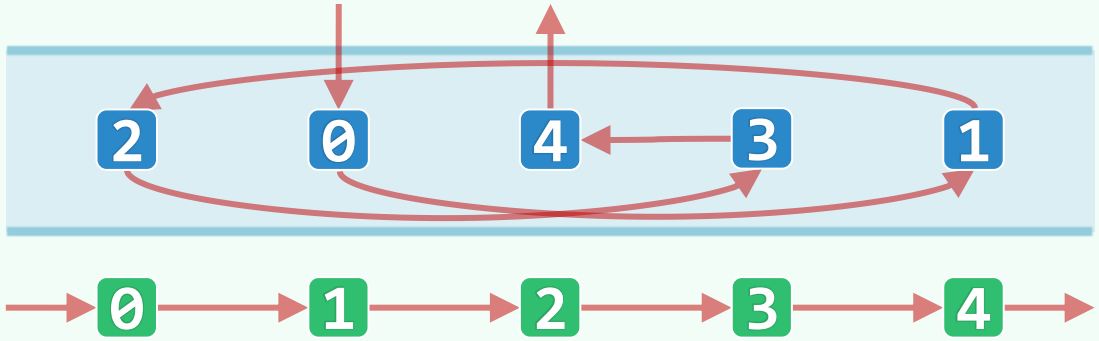
❖ 为保证能够访问到每个元素

- 每一对逻辑上的紧邻元素，都需要彼此记录指向对方的链接|link，即物理位置
- 将链接抽象为元素的位置|position，我们便可以...

循位置访问

- ...循着位置，在常数时间内从一个元素转入下一个

  从而可以抵达列表中的任何一个元素



- 这正如你借助名片

  由某位朋友找到他的某位亲戚，进而他的同事、同事的战友、战友的同学、...

- Call-By-Position：借助元素的位置顺藤摸瓜，找到任一目标元素，如此

  - 物理位置得以独立于逻辑次序，动态操作仅会涉及局部的常数个元素而不再与全局相关

  - 为实现高效率的动态操作提供了机制上的可能与保证

- 我们很快就会看到，列表的多数动态操作都可以在常数时间内完成！

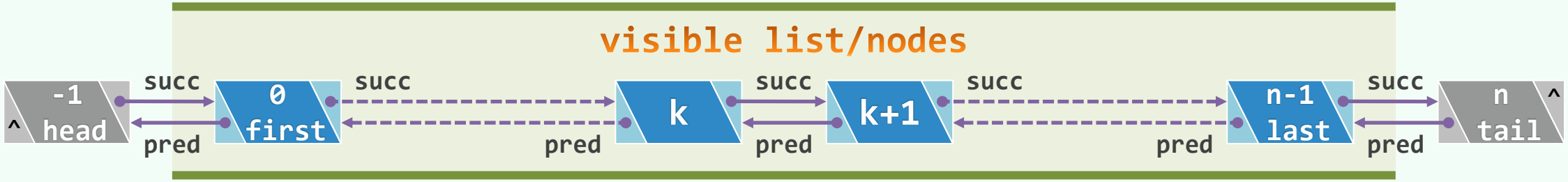
  如何具体实现呢？

链表+节点

❖ 链表|linked list：列表的典型实现方式，分两级封装而成

❖ 对应于列表的元素，链表的基本单位为节点|node

- 既封装了元素所对应的数据项：data
- 也以指针的形式记录了前驱、后继节点的位置：pred|succ



❖ 于是，节点之间可通过指针相互索引、顺次访问

而为了引入新节点或删除原有节点，也只需在局部调整少量相关节点之间的指针 //有望O(1)

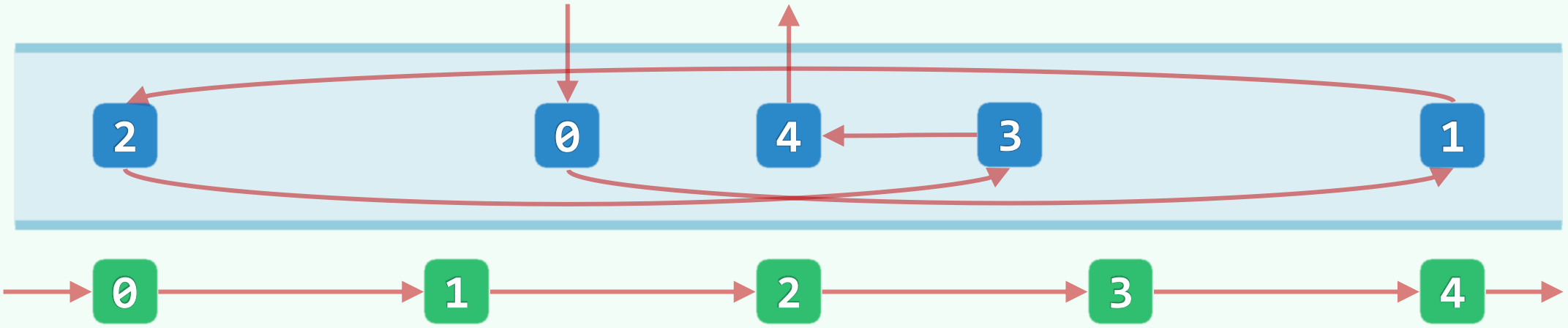
列表

ADT：接口与实现

邓俊辉
deng@tsinghua.edu.cn

ListNode ADT

操作接口	功能
data	节点所存数据对象
pred | succ	前驱|后继节点的位置
insertPred(e) | insertSucc(e)	插入前驱|后继节点
movePred(s) | moveSucc(p)	将当前节点转移至紧靠于s之前|紧接于p之后



ListNode

```
template <typename T> using LNP = ListNode<T>*; //List Node Position

template <typename T> struct ListNode { //简洁起见，完全开放而不再严格封装

   T data;  LNP<T> pred, succ; //数值、前驱、后继

   ListNode() : data(), pred( nullptr ), succ( nullptr ) {}

   ListNode( const T& e, LNP<T> p = nullptr, LNP<T> s = nullptr )

      : data(e), pred(p), succ(s) {} //类T须已定义复制方法

   LNP<T> insertPred( const T& e ); //前插入

   LNP<T> insertSucc( const T& e ); //后插入

   void movePred( LNP<T> s ); //转移至s之前驱

   void moveSucc( LNP<T> p ); //转移至p之后继

};
```



List ADT

操作接口	功能
size() | empty()	报告节点总数|判定是否为空
first() | last()	返回首|末节点的位置
insertFirst(e) | insertLast(e)	将e当作首|末节点插入
insert(p,e) | insert(e,p)	将e当作节点p的后继|前驱插入
remove(p)	删除节点p
sort()	排序
find(e)	在无序列表中，查找与e相等者（如有多个，取最靠前者）
search(e)	在有序列表中，查找不大于e的最大者
dedup()	从无序列表中，剔除相等的元素（保留最靠后者）
uniquify()	从有序列表中，剔除相等的元素（保留最靠前者）
traverse(visit())	遍历列表，统一按visit()处理所有节点

List

```
#include "listNode.h" //引入列表节点类

template <typename T> class List { //列表模板类

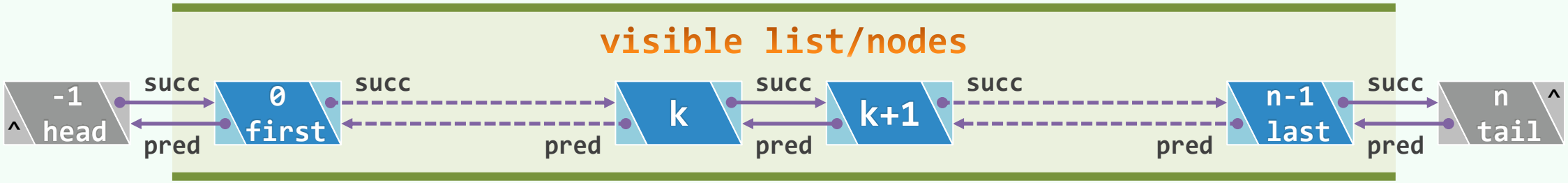
   protected:  Rank _size; LNP<T> head, tail; //哨兵

               //头、首、末、尾节点的秩，可分别理解为-1、0、n-1、n

               /* ... 内部函数 */

   public:     /* ... 构造函数、析构函数、只读接口、可写接口、遍历接口 */

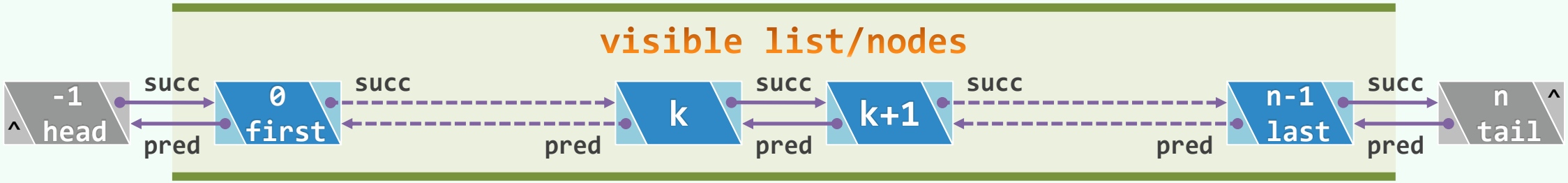
};
```



List: 初始化

```
template <typename T> void List<T>::init() { //初始化，创建列表对象时统一调用
   head = new ListNode<T>;
   tail = new ListNode<T>;
   head->succ = tail;
   tail->pred = head;
   _size = 0;
}
```

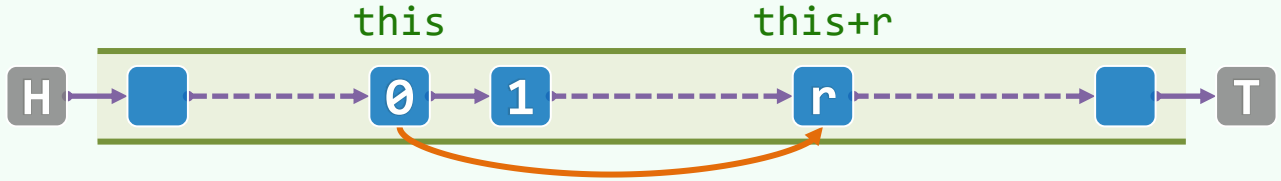




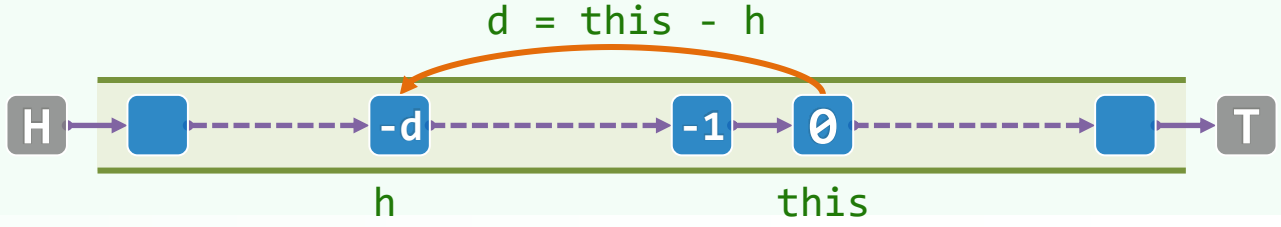
ListNode：模仿循秩访问

```
❖ template <typename T> LNP<T> List<T>::operator[]( Rank r ) const
     { return *first() + r; } //O(r)效率，虽方便，勿多用

❖ template <typename T> LNP<T> ListNode<T>::operator+( Rank r ) {
     for ( LNP<T> p = this; ; p = p->succ, r-- ) //从当前节点出发
        if ( 0 == r ) return p; //顺数第r个节点即是目标节点
  } //秩 == 前驱的总数
```




```
❖ template <typename T> Rank ListNode<T>::operator-( LNP<T> h ) { //h<=this
     for ( Rank d = 0; ; h = h->succ, d++ ) //区间内的节点，挨个报数
        if ( h == this ) return d;
  } //O(n)
```



03.List

B-无序列表

列表

无序列表：插入、删除与移动

邓俊辉
deng@tsinghua.edu.cn

List插入接口

```
❖ LNP<T> List<T>::insert( const T& e, LNP<T> p )
    { _size++; return p->insertPred( e ); } //e当作p的前驱插入

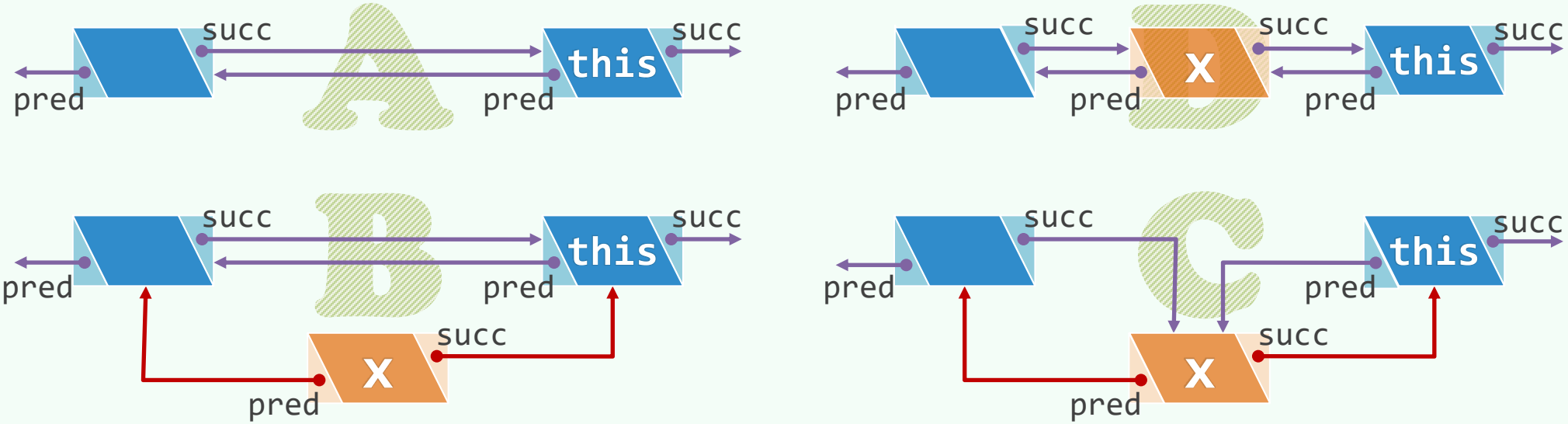
❖ LNP<T> List<T>::insert( LNP<T> p, const T& e )
    { _size++; return p->insertSucc( e ); } //e当作p的后继插入

❖ LNP<T> List<T>::insertFirst( const T& e )
    { _size++; return head->insertSucc( e ); } //e当作首节点插入

❖ LNP<T> List<T>::insertLast( const T& e )
    { _size++; return tail->insertPred( e ); } //e当作末节点插入
```

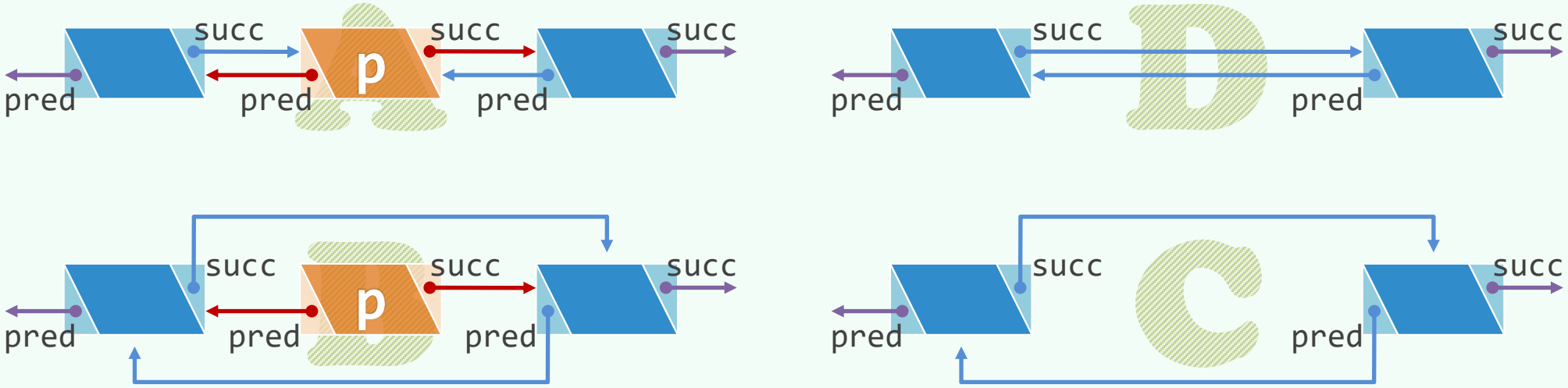
ListNode插入算法

```
template <typename T> LNP<T> ListNode<T>::insertPred( const T& e ) {
   LNP<T> x = new ListNode( e, pred, this ); //创建新节点（pred或为head）
   pred->succ = x; pred = x; return x; //列表接纳新节点（操作次序不可颠倒）
}
```



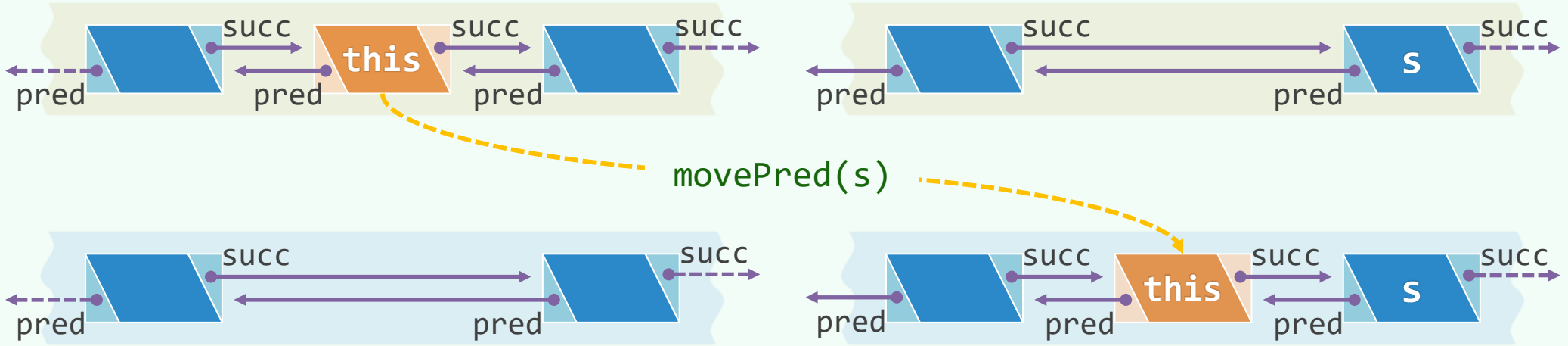
List删除算法

```
template <typename T> T List<T>::remove( LNP<T> p ) {
   T e = p->data; //备份数据项（类型T可直接赋值）
   p->pred->succ = p->succ; p->succ->pred = p->pred; //pred|succ或为head|tail
   delete p; _size--; return e; //释放节点，更新规模，返回数据项
}
```



ListNode移动

```
template <typename T> void ListNode<T>::movePred( LNP<T> s ) { //s != this
   pred->succ = succ; succ->pred = pred; //取出当前节点
   pred = s->pred; pred->succ = this; succ = s; s->pred = this; //转移至紧靠于s之前
} //moveSucc(p)完全对称，O(1)
```



列表

无序列表：复制与析构

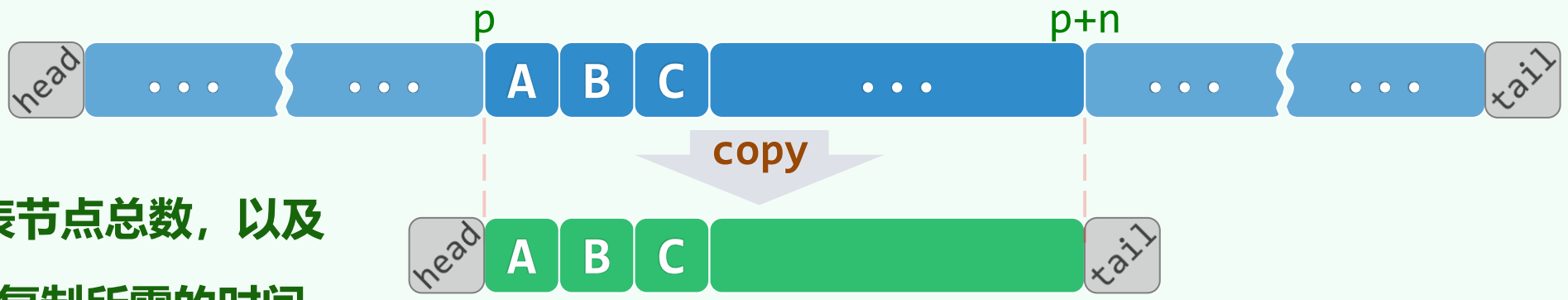
邓俊辉
deng@tsinghua.edu.cn

copy()|深复制

❖ deep copy: 递归地复制蓝本中的每一层嵌套结构

❖ void List<T>::copy(LNP<T> p, Rank n) { //依据蓝本区间，复制出当前列表

```
    for ( init(); n--; p = p->succ ) //先创建空表，再将蓝本区间内各节点
        insertLast( p->data ); //依次复制，并作为末节点插入，O(1)
}
```



❖ 所需的时间

- 线性正比于列表节点总数，以及
- 每个T对象的深复制所需的时间

❖ List<T>::List(List<T> const& L, Rank r, Rank n) { copy(L[r], n); } //区段或

List<T>::List(List<T> const& L) { copy(L.first(), L._size); } //整体深复制

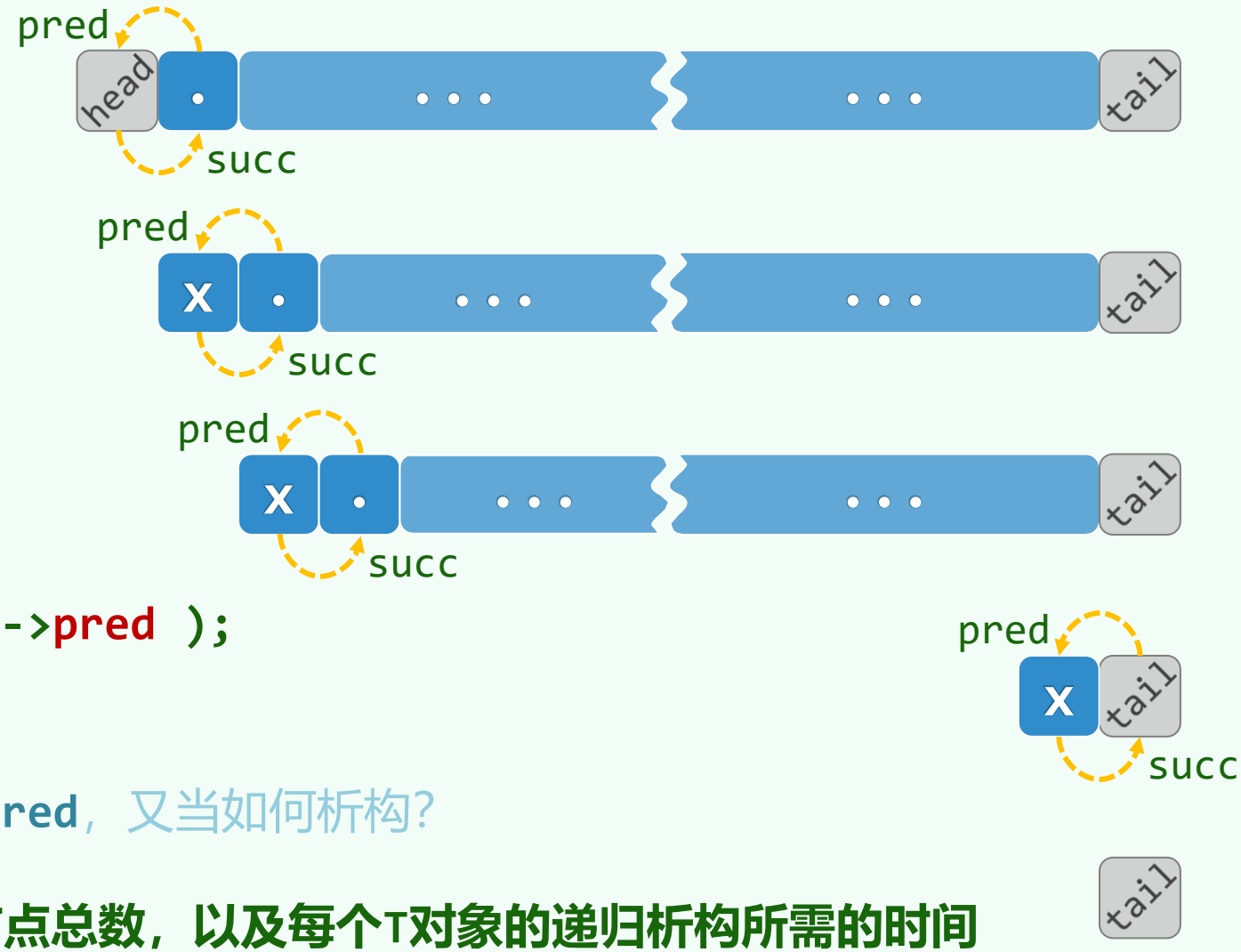
clean()|析构

❖ 反过来，在析构时

也应递归地释放每一层的数据

❖

```
template <typename T>
List<T>::~List() {
   for ( LNP<T> x = head;
         x != tail;
         delete (x = x->succ)->pred );
   delete tail;
} //若是单向链表，只有succ没有pred，又当如何析构？
```



❖ 同理，所需的时间线性正比于列表节点总数，以及每个T对象的递归析构所需的时间

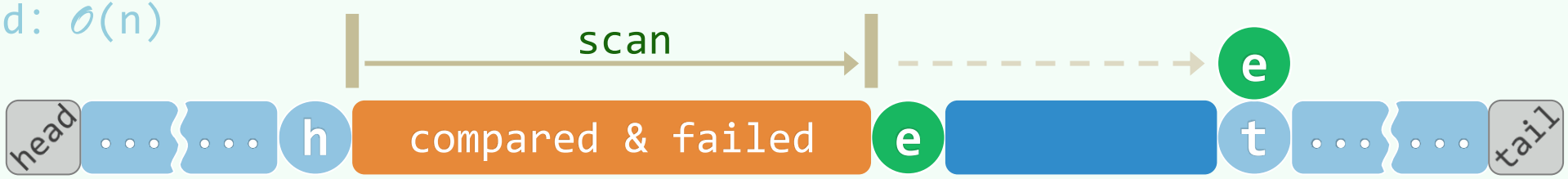
列表

无序列表：查找与去重

邓俊辉
deng@tsinghua.edu.cn

查找

```
template <typename T> LNP<T> //在无序列表区间(h,t)内，找到等于e的最靠前者
List<T>::find( LNP<T> h, const T& e, LNP<T> t ) const { //h < t
   T bak = t->data; t->data = e; //预置哨兵
   while ( e != ( h = h->succ )->data ); //自前向后逐个比对，直至
   t->data = bak;  return h == t ? nullptr : h; //命中或出界
} //find: O(n)
```




```
template <typename T>
LNP<T> find( const T& e ) const { return find( head, e, tail ); }
```

去重

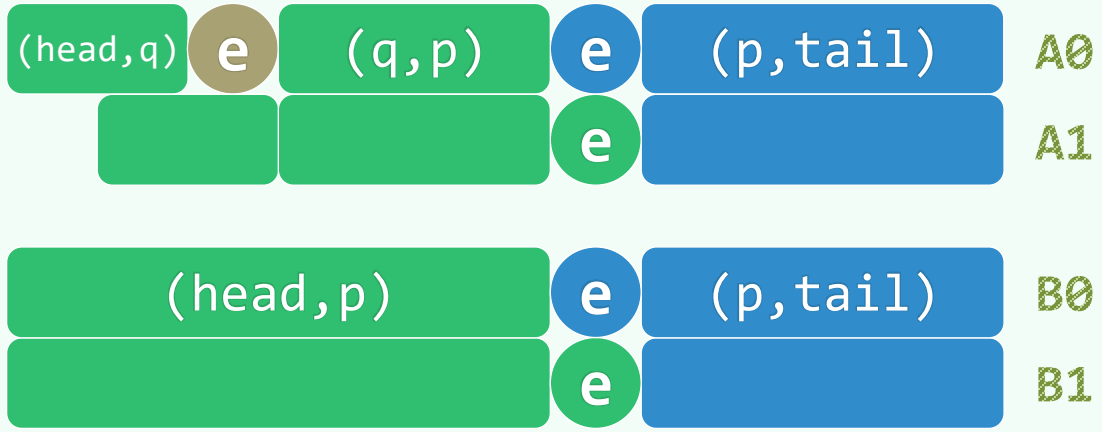
```
❖ template <typename T> void List<T>::dedup() { //O(n^2)
     for ( LNP<T> p = first(); p != tail; p = p->succ ) //O(n)
        if ( LNP<T> q = find( head, p->data, p ) ) //O(n)
           remove( q ); //O(1)
  }
```

❖ 最坏情况（比如互异），$O(n^2)$

与Vector::dedup()一样

❖ 最好情况（比如全等），只需$O(n)$

得益于摆脱了循秩访问的窠臼



列表

无序列表：遍历

邓俊辉
deng@tsinghua.edu.cn

traverse(): 函数指针/函数对象

```
❖ template <typename T>
  void List<T>::traverse( void ( * visit )( T& ) ) {
     for ( LNP<T> p = head->succ; p != tail; p = p->succ )
        visit( p->data );
  }
```




```
❖ template <typename T> template <typename VST>
  void List<T>::traverse( VST& visit ) {
     for ( LNP<T> p = head->succ; p != tail; p = p->succ )
        visit( p->data );
  }
```

实例：统一地将列表中的元素各自加一

❖ 先实现一个类（结构），可使单个T类型的元素加一

```
template <typename T> //假设T可直接递增或已重载操作符“++”
struct Increase //函数对象：通过重载操作符“()”实现
   { void operator()( T& e ) { e++; } }; //加一
```

❖ 再将其作为参数，传递给遍历算法

```
template <typename T> void increase( List<T>& L )
   { L.traverse( Increase<T>() ); } //即可以之作为基本操作，遍历列表
```

❖ 课后练习：模仿此例，实现统一的减一、加倍、求和等更多的遍历功能

实例：统计元素总和，作为校验值

```
❖ template <typename T> struct Cumulate { //累计一个T类型对象的数值
      T& s;
      Cumulate( T& sum ) : s( sum ) {}
      void operator() ( T & e ) { s += e; } //约定T可直接相加
  }; //Cumulate
❖ template <typename T> T sum( List<T>& L ) { //统计列表中所有元素的总和
      T sum = 0;
      L.traverse( Cumulate<T>( sum ) ); //以Cumulate为基本操作进行遍历
      return sum; //返回总和
  } //sum
```

实例：统计逆序对总数，判断是否有序

```
❖ template <typename T> struct Compare { //判断线性序列中的一个元素是否与其前驱顺序
     T pred; int& inv; //前驱、（相邻）逆序对的计数器
     Compare( int& inversions, T& first ) : pred( first ), inv( inversions ) {}
     void operator() ( T & e ) { inv += ( pred > e ); pred = e; }
  }; //Compare
❖ template <typename T> int inv( List<T>& L ) { //判断列表是否整体有序
     int inversions = 0;
     L.traverse( Compare<T>( inversions, L[0]->data ) ); //以Compare为基本操作做遍历
     return inversions; //返回总数
  } //inv
```

03.List

C-有序列表

列表

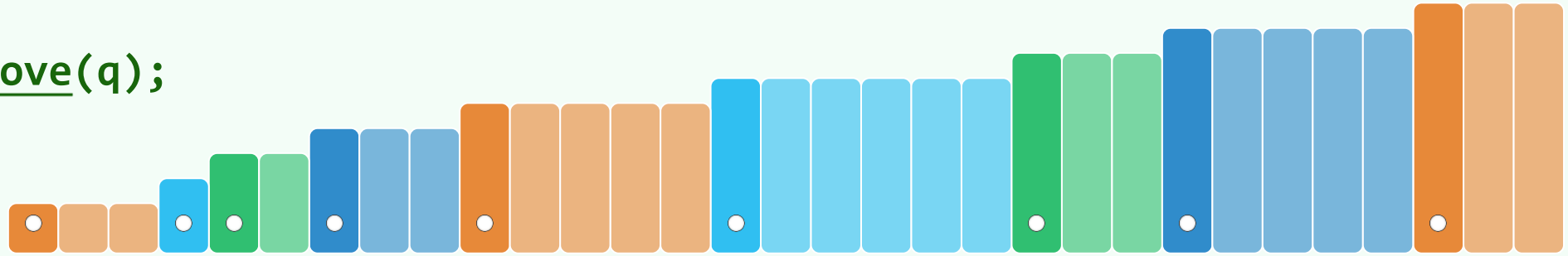
有序列表：去重与查找

邓俊辉
deng@tsinghua.edu.cn

去重

❖ 摒弃循秩访问转而循位置访问，使得动态操作只需常数时间，整体自然达到线性效率

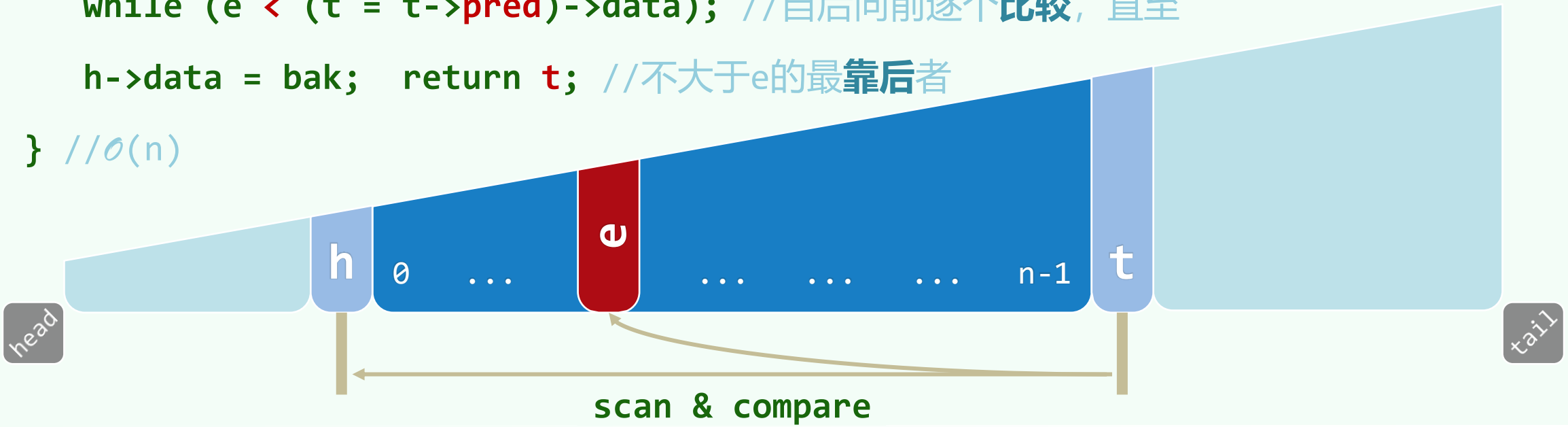
❖
```
template <typename T> void List<T>::uniquify() { //只需遍历列表一趟，O(n)
   if (_size < 2) return; //过短的列表，自然不含相等元素
   for (LNP<T> p = first(), q = p->succ; q != tail; q = p->succ)
      if (p->data != q->data) //若p与其后继q不等
         p = q; //则前进一步
      else //否则直接删除后者，而不必如向量那样间接删除
         remove(q);
}
```



查找

❖ 凡事一利一弊：动态操作性能提高的同时，静态操作性能可能反降

❖
```
template <typename T> LNP<T> //有序列表区间(h,t)：h < t
List<T>::search(LNP<T> h, const T & e, LNP<T> t) const { //等效于h->data == -inf
   T bak = h->data;  h->data = e; //预置哨兵
   while (e < (t = t->pred)->data); //自后向前逐个比较，直至
   h->data = bak;  return t; //不大于e的最靠后者
} //O(n)
```



03.List

D-选择排序

列表

选择排序

邓俊辉
deng@tsinghua.edu.cn

排序器：统一入口

```
template <typename T> void List<T>::sort( LNP<T> h, LNP<T> t ) {
   switch ( rand() % 4 ) {
      case 1 :  insertionSort( h, t ); break; //插入排序
      case 2 :  selectionSort( h, t ); break; //选择排序
      case 3 :      mergeSort( h, t ); break; //归并排序
      default :     radixSort( h, t ); break; //希尔排序（第14章）
   } //随机选择算法，以尽可能充分地测试；应用时可视具体问题的特点，灵活确定或扩充
} //sort
```

起泡排序：温故知新

❖ 每趟扫描都需 $\mathcal{O}(n)$ 次比较、$\mathcal{O}(n)$ 次交换

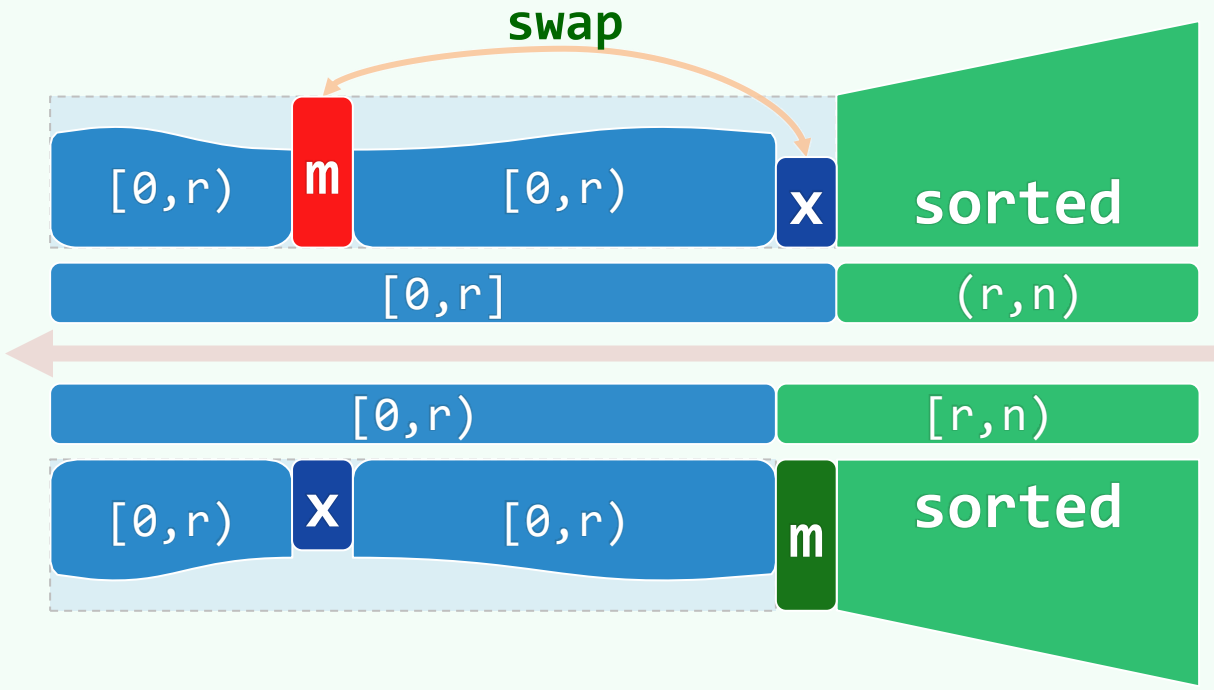
然而，其中的 $\mathcal{O}(n)$ 次交换完全没有必要

❖ 每趟扫描的实质效果无非是

- 通过比较找到当前的

  最大元素m，并

- 通过交换使之就位

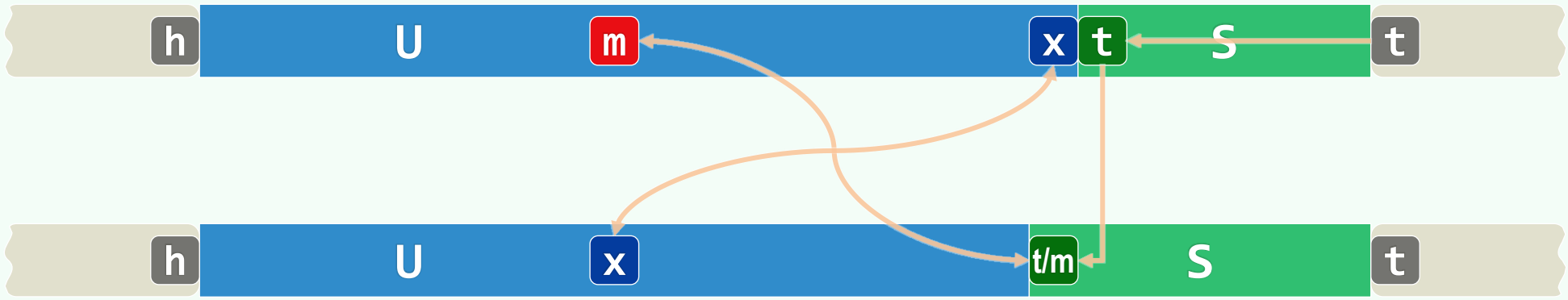


❖ 因此实际上，

在经 $\mathcal{O}(n)$ 次比较确定m之后，仅需一次交换即足矣

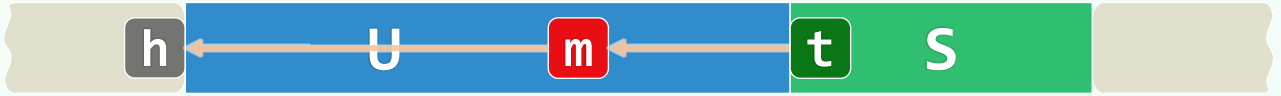
算法实现

```
❖ template <typename T> void List<T>::selectionSort( LNP<T> h, LNP<T> t ) { //h < t
    for ( LNP<T> x = t->pred; h->succ != x; t = x, x = t->pred ) //从无序前缀(h,x]中
        swap( getMax(h,t)->data, x->data ); //找出最大者，并通过交换令其就位
        //getMax(h,t)->movePred(t); //亦可通过移动就位：同样是O(1)成本，且能保证稳定性
}
```



getMax(): 自后向前 + 严格比较 = 稳定性

```
❖ template <typename T> LNP<T> List<T>::getMax( LNP<T> h, LNP<T> t ) { //2 <= t-h
    LNP<T> m = t->pred; //当前的最大者
    for ( t = m->pred; h != t; t = t->pred ) //自后向前，逐个比较
       if ( m->data < t->data ) //见贤：若发现严格更大的元素
          m = t; //思齐：则更新记录
    return m; //最终的最大者：即便有多个，也总是取最靠后者
 } //getMax, O(n)
```



性能分析

- 共n步迭代：swap()累计$\mathcal{O}(n)$，getMax()累计$\mathcal{O}(n^2)$

  对任何输入序列，getMax()累计耗时都是$\Theta(n^2)$ ——没有最好情况！

- 实际上，$\Theta(n^2)$来自比较操作；移动操作只需$\Theta(n)$次，远少于起泡排序

  就常系数而言，移动操作远大于比较操作，对实际性能影响更大

- 可否...每轮只做 $o(n)$ 次比较，即找出当前的最大元素?

  可以！...利用高级数据结构，getMax()可改进至 $\mathcal{O}(\log n)$

  当然，如此立即可以得到$\mathcal{O}(n \cdot \log n)$的排序算法

- 最大元素m有可能已经就位，此时的swap()显得多余 <-- 可否避免这类徒劳无益的原地交换?

  为此，我们需要切换到一个新的视角，以重新审视算法的过程...

列表

选择排序：轮换

邓俊辉
deng@tsinghua.edu.cn

Permutation|Cycle

❖ 考查由n个互异元素构成的集合$\mathcal{X}$，将其对应的有序序列记作 $S[0,n)$

❖ 由$\mathcal{X}$排成的每个序列 $A[0,n)$

都对应于 S 的一个排列 $\tau()$：

对 $k \in [0,n)$，有 $S[\tau(k)] = A[k]$

k:	0	1	2	3	4	5	6	7	8	9	10	11	12	13	14	15
S:	A	B	C	D	E	F	G	H	I	J	K	L	M	N	O	P
A:	J	N	P	M	A	I	G	O	D	C	H	B	K	L	F	E
$\tau(k)$:	9	13	15	12	0	8	6	14	3	2	7	1	10	11	5	4

❖ 若 $\tau^{(d)}(k) = k$，则 $\langle\, k = \tau^0(k),\ \tau^1(k),\ \tau^2(k),\ \tau^3(k),\ \ldots,\ \tau^{(d-1)}(k) \,\rangle$ 构成一个长度为 d 的轮换

❖ 任何排列，都可分解为若干轮换

- 最多n个
- 最少1个

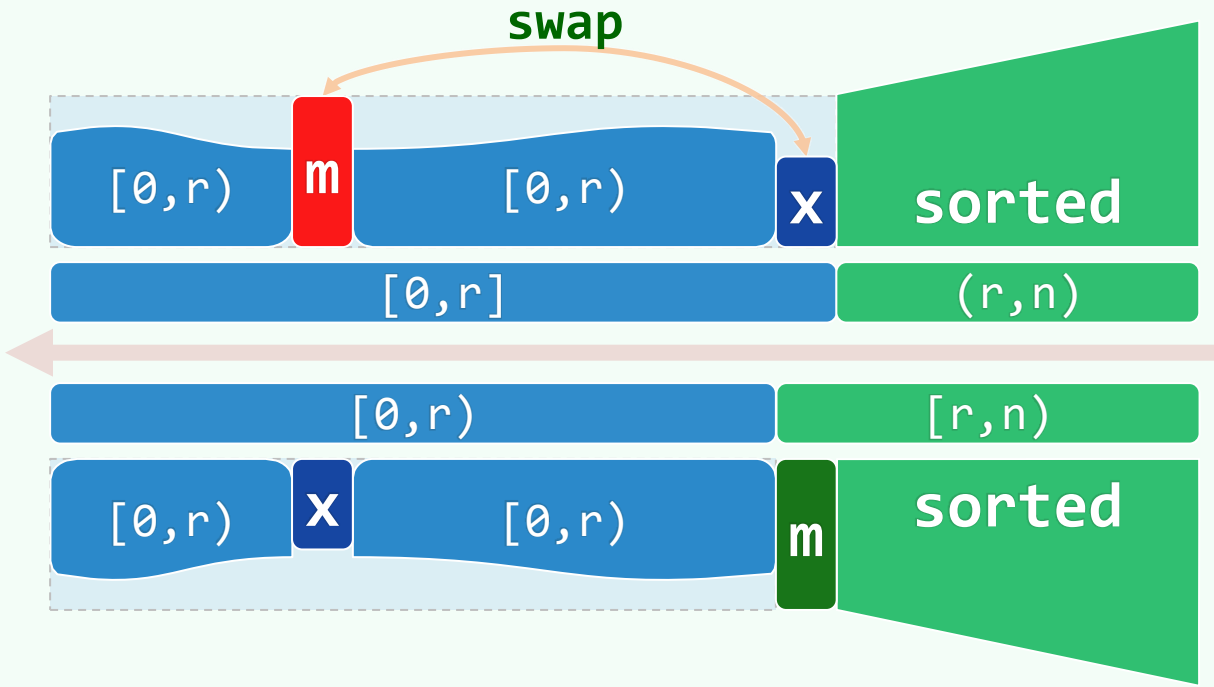
0	9	2	15	4	1	13	11	3	12	10	7	14	5	8	6
A	J	C	P	E	B	N	L	D	M	K	H	O	F	I	G
J	C	P	E	A	N	L	B	M	K	H	O	F	I	D	G
9	2	15	4	0	13	11	1	12	10	7	14	5	8	3	6

swap|cycle

❖ 交换之前：m与x同属一个轮换C，且前后紧邻

❖ 交换之后：

- m从C中逸出，并自成一个轮换
- C的长度减一（可能归零）
- 其余的轮换，保持不变

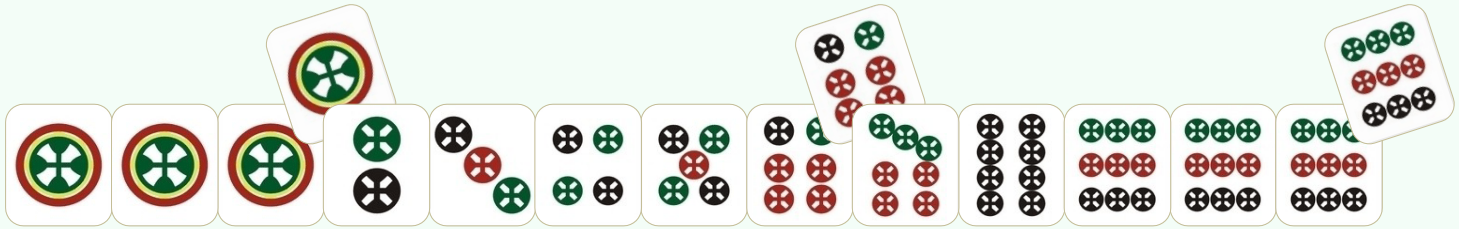


❖ C含k个轮换，就会出现k次原地的交换

固然有 $1 \le k \le n$，但数学期望呢?

❖ 原地的交换 ~ C仅含m ~ m已经就位 ~ x=m

- $\mathbb{E}(k) = 1/1 + 1/2 + 1/3 + \cdots + 1/n = \Theta(\log n)$



列表

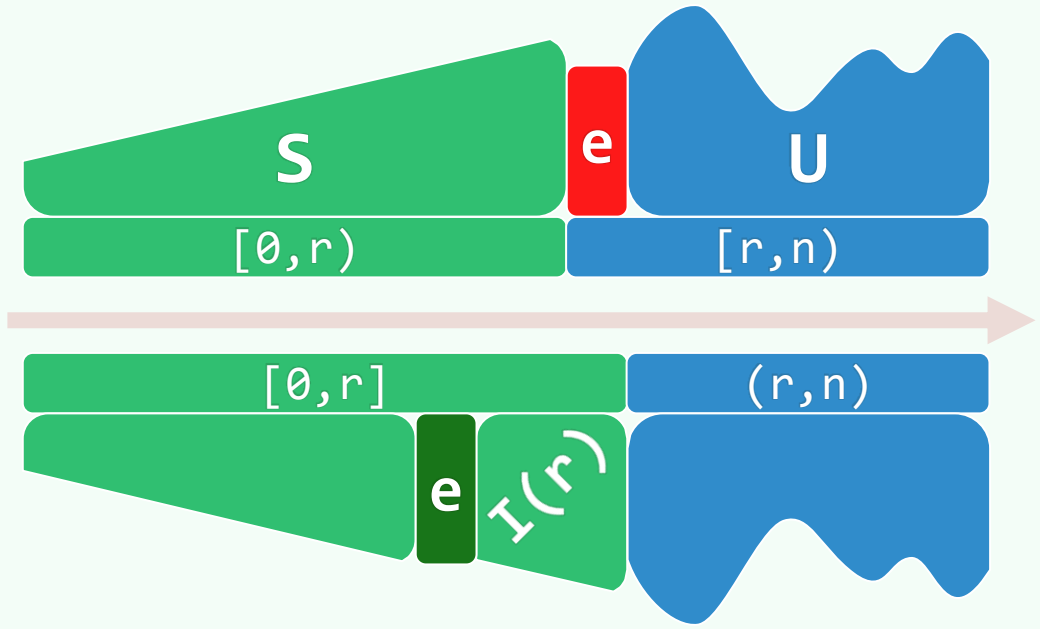
插入排序：算法与实现

03.List

E-插入排序

邓俊辉
deng@tsinghua.edu.cn

减而治之



❖ 始终将序列视作两部分：

- 前缀 S[0,r)：有序
- 后缀 U[r,n)：待排序

❖ 初始化：|S| = r = 0

❖ 反复地，针对e = A[r]

- 在S中查找适当位置
- 插入e
- r ++

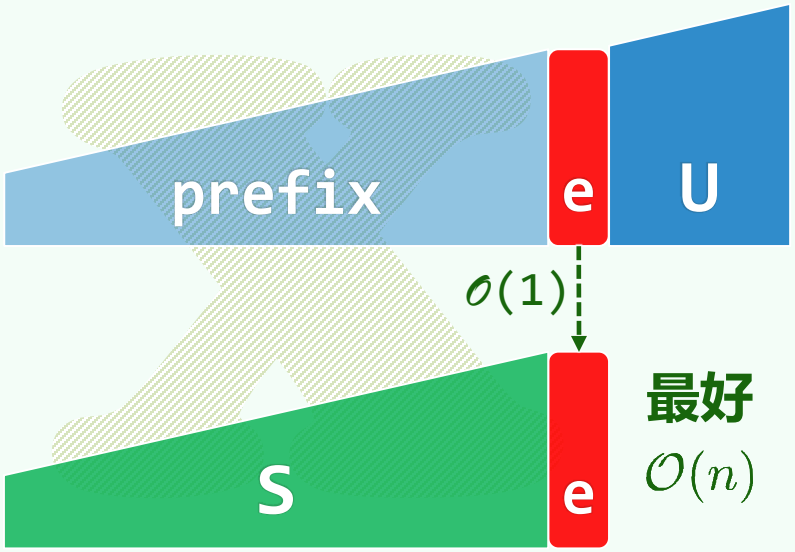
算法实现

```
template <typename T> //逐一引入节点r，持续拓展有序前缀(h,r)
void List<T>::insertionSort( LNP<T> h, LNP<T> t ) {
   for ( LNP<T> r = h->succ, s = r->succ; r != t; r = s, s = r->succ ) //反复地
      r->moveSucc( search( h, r->data, r ) ); //将[r]转移至(h,r)内的适当位置
} //仅使用O(1)辅助空间，属于就地算法
```

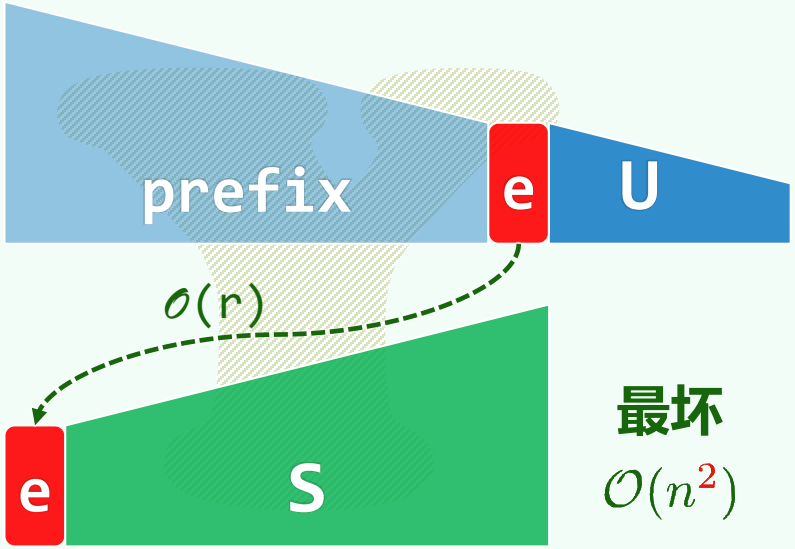
❖ 得益于此前约定的search()接口语义，前缀的确总是保持有序，而且稳定

❖ 验证以下情况，体会哨兵的作用：前缀中有元素与r相等；r在前缀中最小/最大；前缀为空；...

复杂度



最好
$\mathcal{O}(n)$



最坏
$\mathcal{O}(n^2)$

❖ 有实质的差异！selectionSort()呢?

❖ 输入敏感性|input-sensitivity

序列有多乱，就花费多少时间

//稍后详解

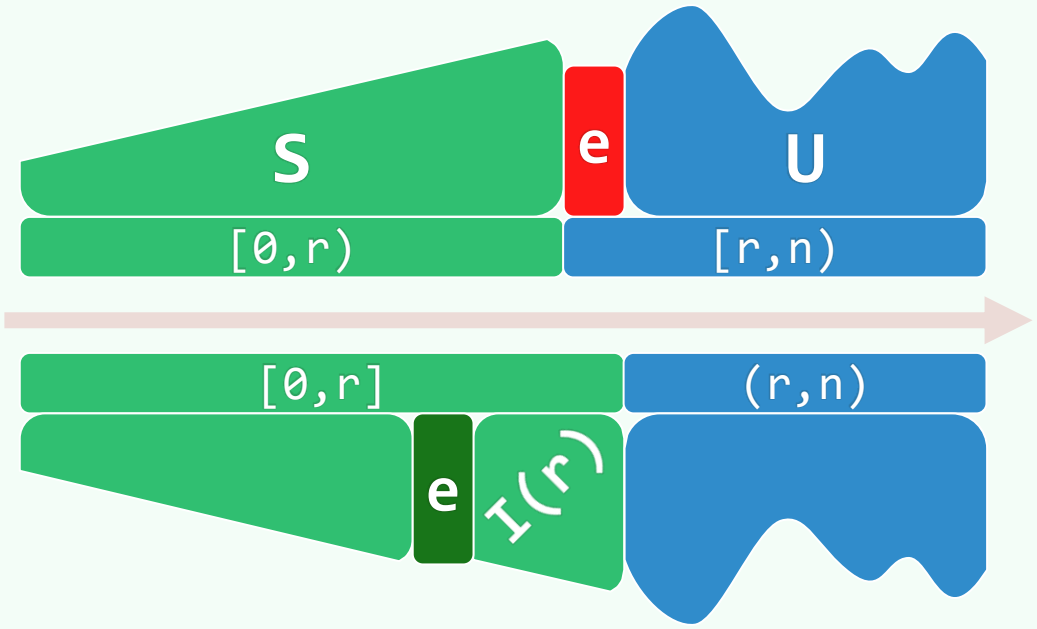
❖ 为插入[r]，期望成本为r/2

总体 = $\sum_{r=0}^{n-1} r/2 = \mathcal{O}(n^2)$

❖ 主要消耗来自查找，且前缀总是有序的

为何不...改用...binSearch()?

在线



- 日常牌戏中，为何你我他都用它?
- 改用其他算法...

  发牌完毕，才能各自开始理牌
- Insertionsort

  可以边发边理；发完即理好
- online

  数据完全就绪之前，即可开始计算!

  //何必一味追求结果，不妨充分享受过程

性能分析

- 属于就地算法（in-place）
- 属于在线算法（online）
- 具有输入敏感性（input sensitivity）
  - 后面将会看到：Shellsort之类算法的高效性，完全依赖于insertionsort的这一特性
- 最好情况：完全（或几乎）有序
  - 每次迭代，只需1次比较，0次交换：累计O(n)时间！
- 最坏情况：完全（或几乎）逆序
  - 第k次迭代，需O(k)次比较，1次交换：累计$O(n^2)$时间！
- “优化”的可能：在有序前缀中的查找定位，为何采用了顺序查找，而不是二分查找？

列表

插入排序：逆序对

邓俊辉
deng@tsinghua.edu.cn

逆序对

❖ 在序列S[0,n)中，若有 $0 \le i < j < n$ 且 $S[i] > S[j]$

则称 $\langle i, j \rangle$ 为一个**逆序对**|inversion

❖ 为便于统计，可将逆序对统一记到**后者**j的账上

$$\mathcal{I}(j) = \| \{ 0 \le i < j \mid S[i] > S[j] \} \|$$

❖ 例：{ 5, 3, 1, 4, 2 } 中，共有 0 + 1 + 2 + 1 + 3 = 7 个逆序对

{ 1, 2, 3, 4, 5 } 中，共有 0 + 0 + 0 + 0 + 0 = 0 个逆序对

{ 5, 4, 3, 2, 1 } 中，共有 0 + 1 + 2 + 3 + 4 = 10 个逆序对

❖ 显然，逆序对总数 $\mathcal{I} = \sum_j \mathcal{I}(j) \le \binom{n}{2} = \mathcal{O}(n^2)$

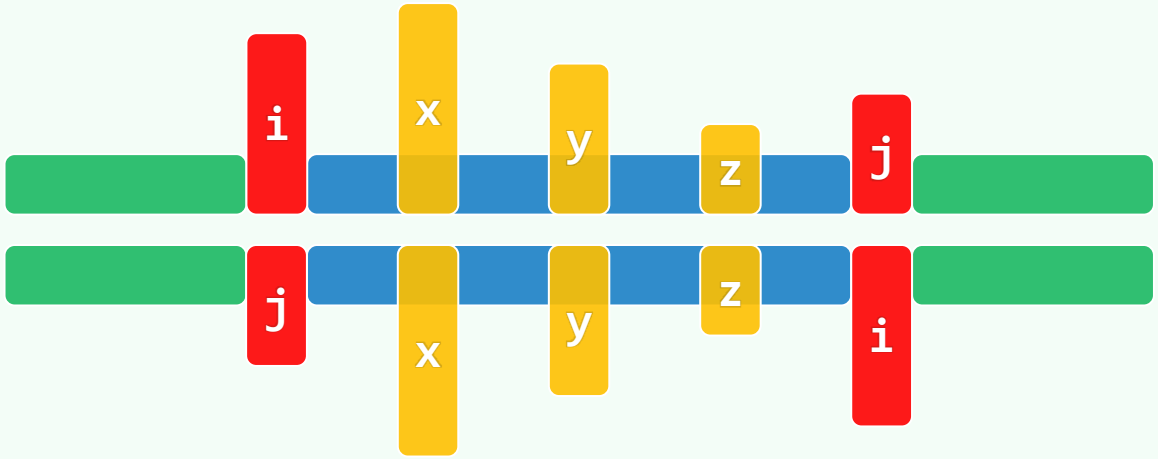
交换与修复

- 交换一对逆序元素，逆序对必然减少

  最少会减少几个？最多呢？

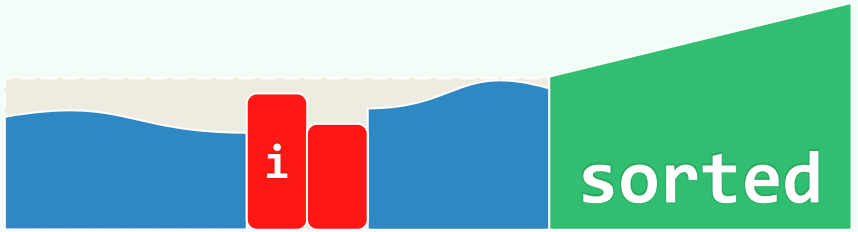
- 通过一次交换，修复相距更远的逆序对

  期望而言，可以同时修复更多的逆序对

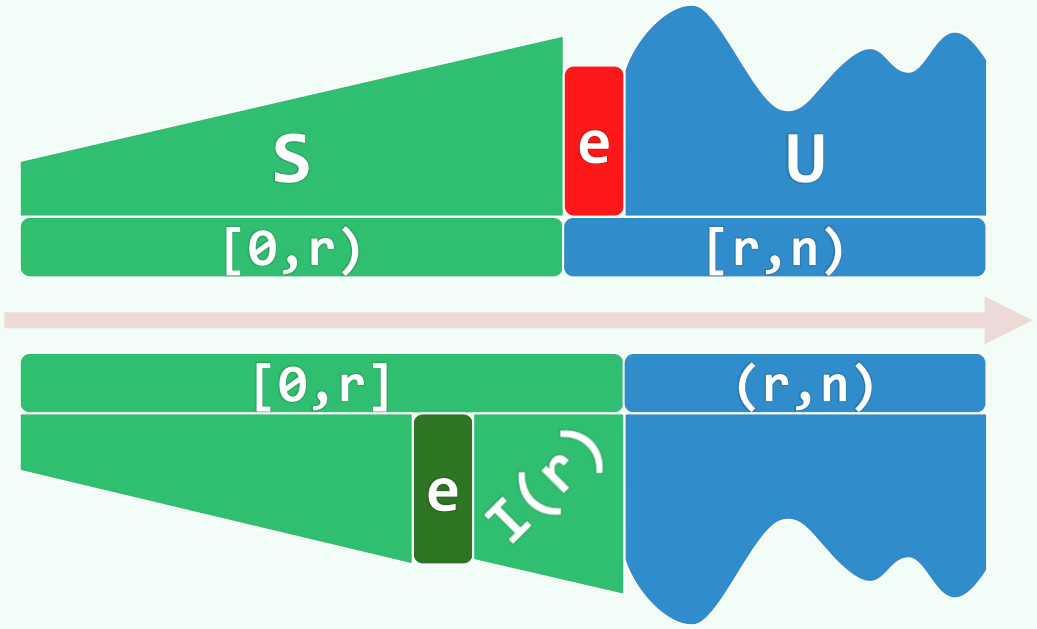


- Bubblesort
  - 每次交换的，都是紧邻的逆序对
  - 逆序对只能逐一地修复

  最坏情况下必然需要$O(n^2)$时间



输入敏感



- 从逆序对的视角，重新审视插入排序...
- 针对 $e = S[r]$ 的那一步迭代

  恰好需做 $\mathcal{I}(r)$ 次元素比较操作
- 若共含 $\mathcal{I}$ 个逆序对，则
  - 比较的次数为 $\mathcal{O}(\mathcal{I})$
  - 运行时间为 $\mathcal{O}(n + \mathcal{I})$
- 序列（含逆序对总数） ~ 衣物（脏污程度）

  插入排序（所需时间） ~ 洗涤（用时长短）

03.List

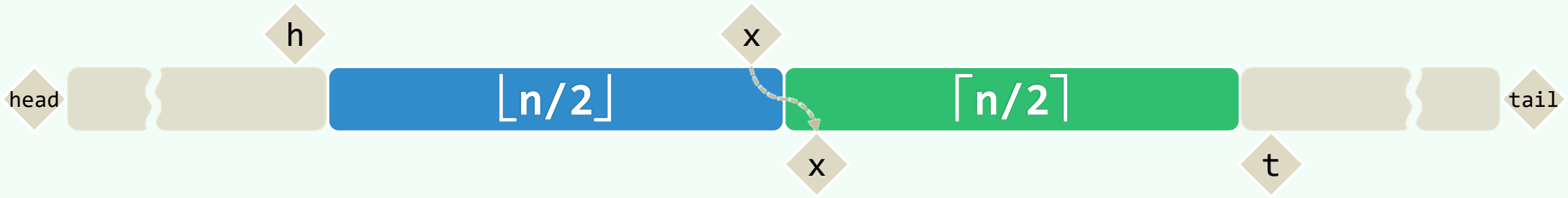
F-归并排序

列表

归并排序：算法与实现

邓俊辉
deng@tsinghua.edu.cn

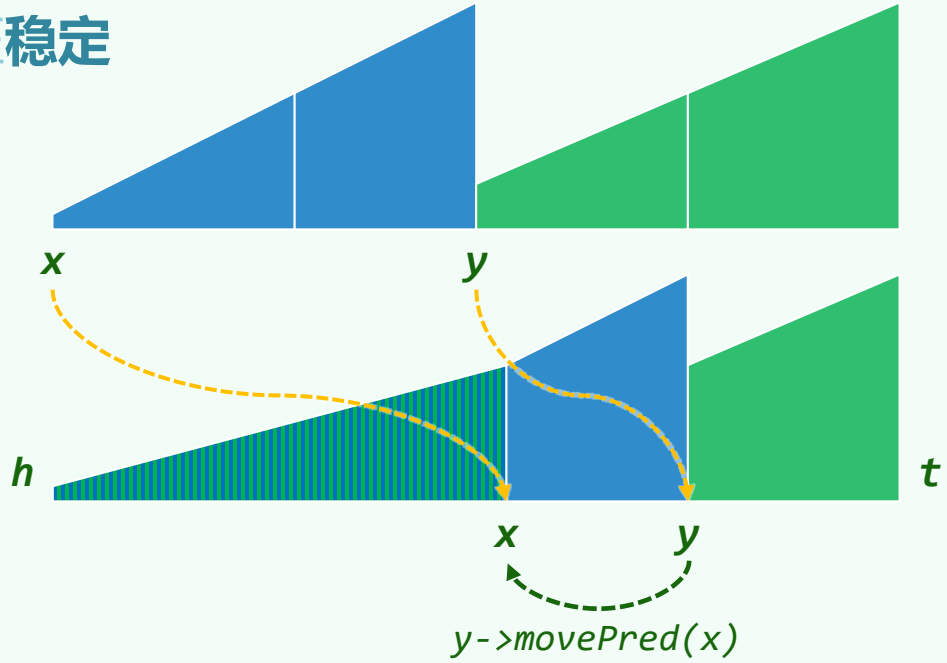
主算法




```
❖ template <typename T> void List<T>::mergeSort( LNP<T> h, LNP<T> t ) { //h < t
     Rank n = *t - h - 1;  if ( n < 2 ) return; //区间长度不足2时，直接返回；否则...
     LNP<T> x = *h + n/2; //切分：(h,t) = (h,x] + (x,t)
     mergeSort( x, t ); mergeSort( h, x = x->succ ); //后缀、前缀各自排序
     merge( h, x, t ); //归并：(h,x) + [x,t)
  } //若归并可在线性时间内完成，则总体运行时间亦为O(nlogn)
```

二路归并 = 增量式插入排序

```
template <typename T> LNP<T> //将[y,t)中的节点，分别转移至(h,y)中的适当位置
List<T>::merge( LNP<T> h, LNP<T> y, LNP<T> t ) {
   for ( LNP<T> x = h->succ; (x != y) && (y != t);  )
      if ( x->data > y->data ) //小者优先，保证稳定
         (y = y->succ)->pred->movePred(x);
      else
         x = x->succ;
} //O(n)，线性正比于节点总数
```



列表

归并排序：逆序对

邓俊辉
deng@tsinghua.edu.cn

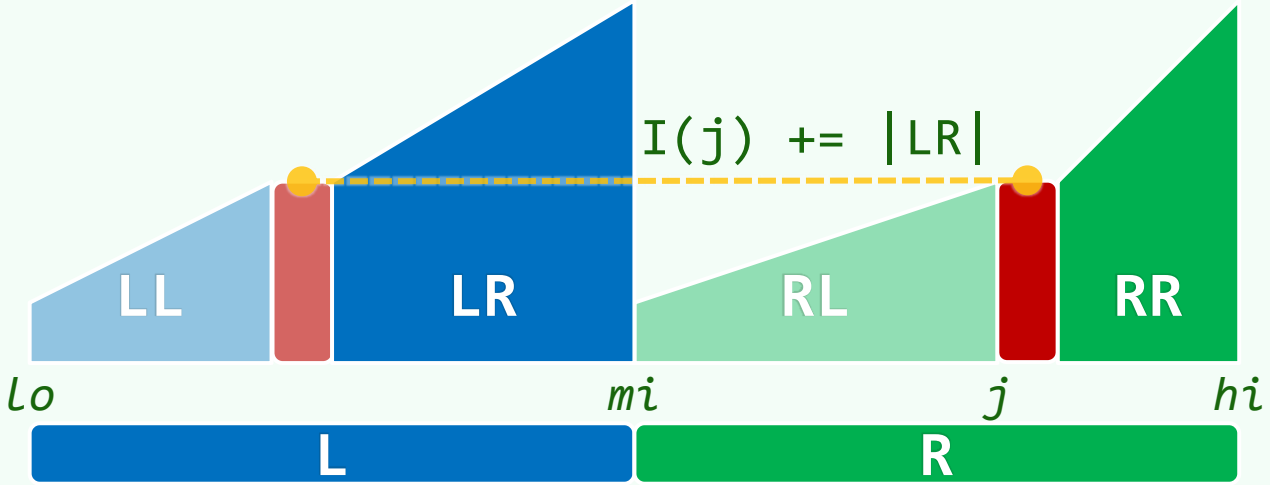
任意给定一个序列，如何统计其中逆序对的总数?

❖ 蛮力算法需要 $\Omega(n^2)$ 时间；而套用归并排序的框架，则仅需 $\mathcal{O}(n \log n)$ 时间...

❖ 分治：L[lo,mi) + R[mi,hi)

- 偏于一侧的逆序对可递归统计
- 实质地，只需考查跨界的逆序对

$\langle i, j \rangle$: $lo \le i < mi \le j < hi$



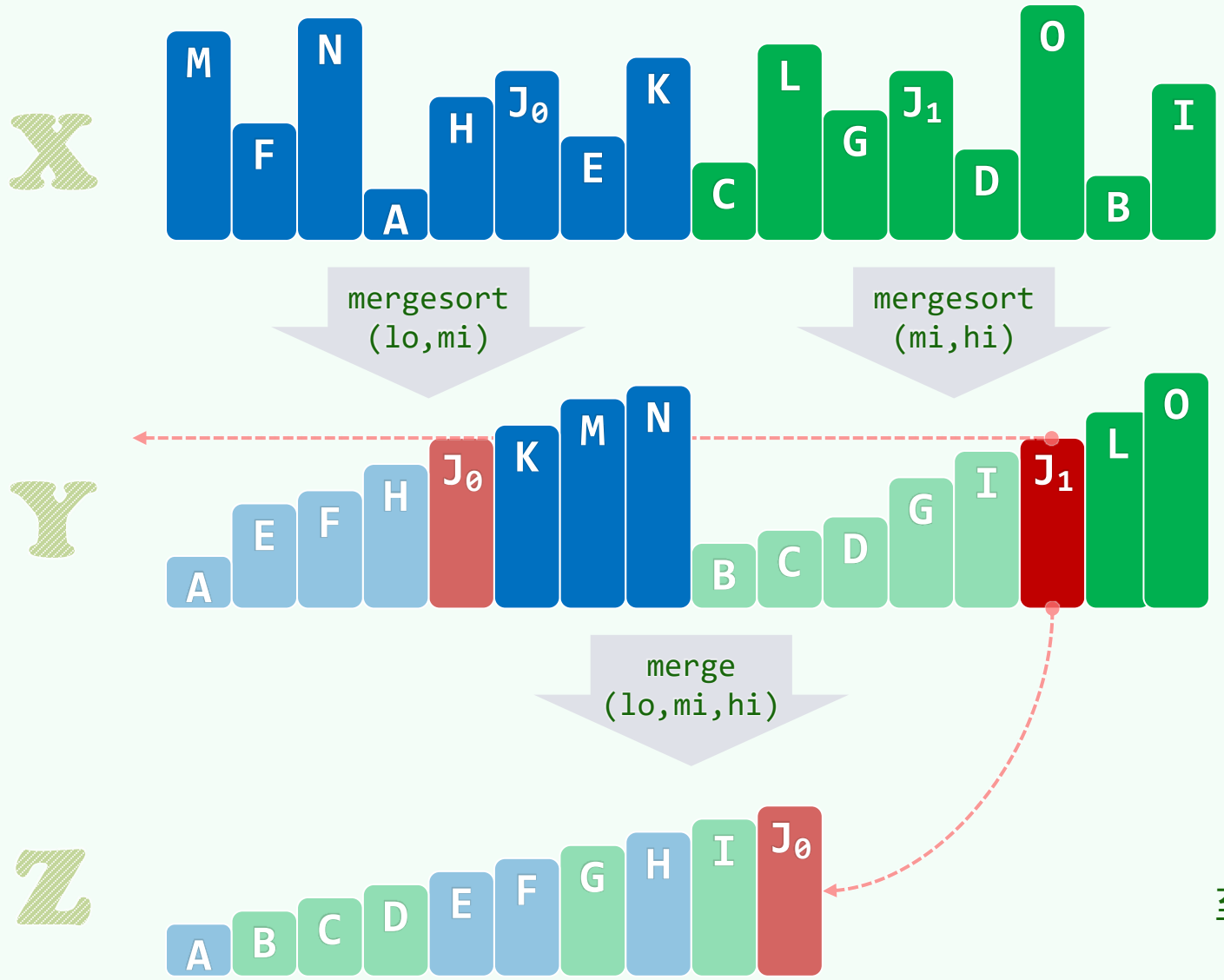
❖ 后一类逆序对，分别记到$[j]$的账上：

$$\hat{\mathcal{I}}(j) = \| \{ lo \le i < mi \mid A[i] > A[j] \} \|$$

❖ 假想着对序列做归并排序，定格于$[j]$刚被选中、即将出列的时刻

此时必有：$\hat{\mathcal{I}}(j) = |LR|$

算法



紧邻的一对子任务/子序列：

若前缀中的MNK与后缀中的J_1逆序

则在子序列各自递归排序之后...

...归并进行至J_1**即将**出列就位时

前缀中必**恰好**只剩下KMN

虽次序或有调整，但总数不变

如此，便可统计出**跨越**前、后缀的逆序对

至于前、后缀各自**内部**的逆序对，则可...